widely in enterprises, with organisations investing heavily across numerous business functions and use cases.

LLMs are easily accessible to the public through interfaces like:

- Anthropic's Claude, Open AI's ChatGPT, Microsoft's Copilot, Meta's Llama models, and Google's Gemini assistant, along with its BERT and PaLM models.
- IBM maintains a Granite model series on watsonx.ai, which has become the generative AI backbone for other IBM products like watsonx Assistant and watsonx Orchestrate.

The four pillars of AI and LLM observability

AI and LLM observability is the continuous practice of capturing, analysing, and evaluation of telemetry data across LLM pipelines and autonomous agents to maintain application performance, accuracy, and safety (Figure 1). Unlike deterministic conventional code, LLM systems are probabilistic and non-deterministic, generating highly unpredictable outputs that make classic logging infrastructure completely insufficient.

Comprehensive observability allows teams to move away from unreliable 'vibe checks' towards production-ready software systems with robust metrics.

While traditional software tracking relies on metrics, logs, and traces (MELT), AI architectures necessitate specialised abstractions.

Traces and spans: Map the exact sequential journey of a request across multi-turn sessions, parallel tool executions, and internal model decision logic.

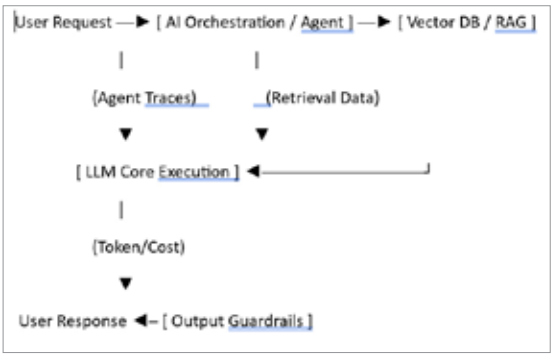

Figure 1: Pillars of AI and LLM

Real-time evaluations (Evals): Employ 'LLM-as-a-judge' algorithms alongside human feedback chains to mathematically score production traffic.

Data surface tracking: Validates data parity across retrieval context (RAG pipelines), prompt engineering templates, and vector database embeddings.

Cost and infrastructure profiling: Calculates hardware strain (GPU/CPU usage) against specific API token counts to precisely optimise financial overhead.

An effective AI observability strategy categorises telemetry into four key operational layers: performance, quality and semantics, cost economics, and security and safety.

Tools selection criteria for AI engineering stack

Tool selection depends heavily on the scale of deployment, data residency constraints, and the framework ecosystem.

- ***LangSmith:*** Best for complex debugging and real-world agent evaluation loops. It provides native integration with LangChain/LangGraph, deep tree tracing, and features annotation queues for subject matter experts to manually grade outputs.
- ***Langfuse:*** Best all-in-one open source choice. This platform offers MIT-licensed, incrementally adoptable tools tracking prompt history, SDK hooks, and user-behaviour analytics side-by-side.
- ***Arize Phoenix:*** Best for local debugging and strict OpenTelemetry (OTel) compliance. It features visual data graphs to map agent reasoning chains and specialised RAG triad evaluations.
- ***Datadog LLM Observability:*** Best for unified enterprise APM tracking. It correlates low-level infrastructure bottlenecks (like cloud server drops) with real-time prompt latency and application errors on a single dashboard.
- ***Helicone:*** Best lightweight gateway proxy solution. It sits directly between your system and LLM providers via a quick URL swap to deliver edge-caching and instant pricing telemetry.

Table 1: Key metrics used to monitor AI and LLM observability

Metric category	Specific signals	Failure/Success indicators
Output quality	Hallucination, Groundedness, Toxicity, PII leakage	High hallucination triggers auto-fallback or guardrail blocks.
RAG and context	Context relevance, Faithfulness, Chunking drift	Irrelevant chunk retrievals directly cause model hallucinations.
Performance	Time-To-First-Token (TTFT), End-to-end latency	Spikes indicate pipeline blockages or model degradation.
Economics	Prompt/completion token split, Cost-per-user	Outliers detect infinite agent loops or bad prompt structures.